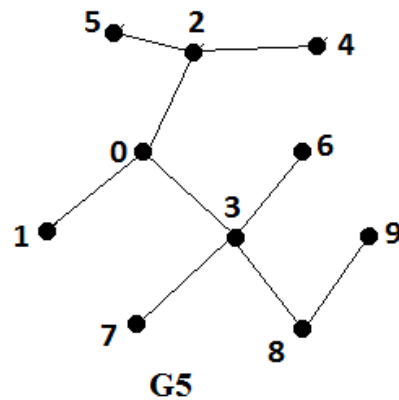
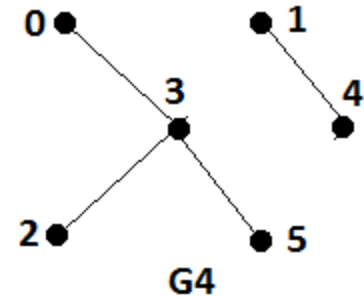
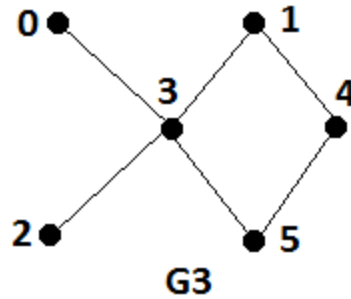
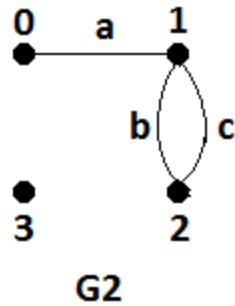
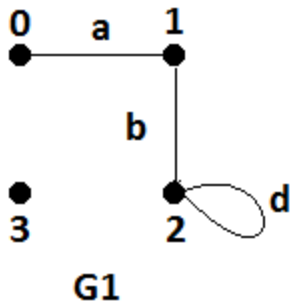


Chương 3. Cây

3.1 Cây :

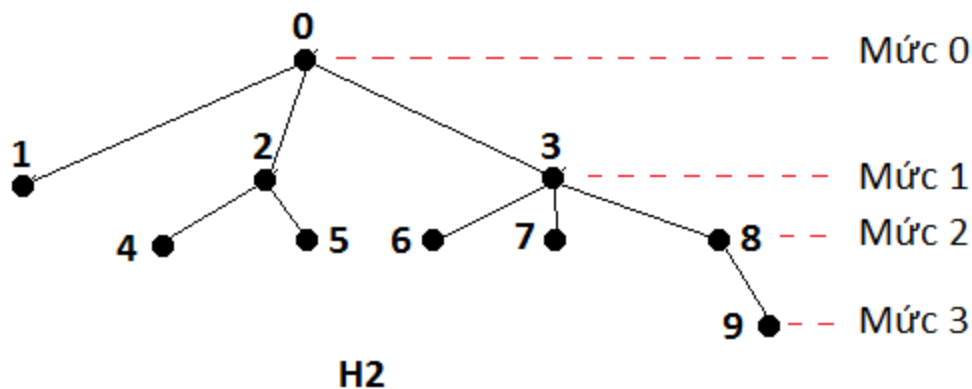
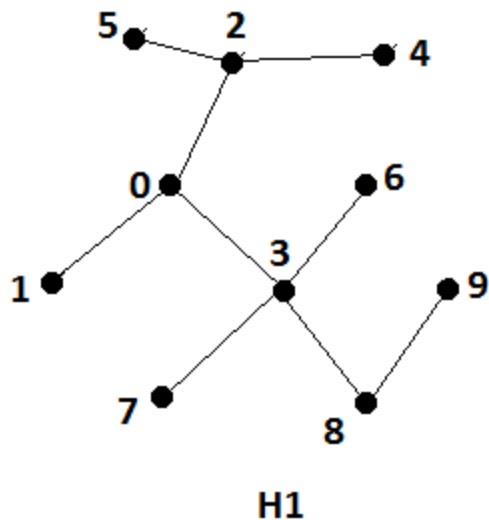
3.1.1 Định nghĩa cây :

Cây T là một **đồ thị đơn giản** thỏa : Nếu v và w là hai đỉnh trong T , thì có một **đường đi sơ cấp** duy nhất nối v và w.



3.1.2 Định nghĩa cây có gốc:

- Cây T có gốc là cây mà trong T một đỉnh v nào đó được chọn làm gốc (duy nhất).
- Độ dài (số cạnh) **đường đi sơ cấp** từ gốc đến đỉnh v được gọi mức của v. Gốc có mức 0.
- Mức cao nhất được gọi là **chiều cao của cây**.

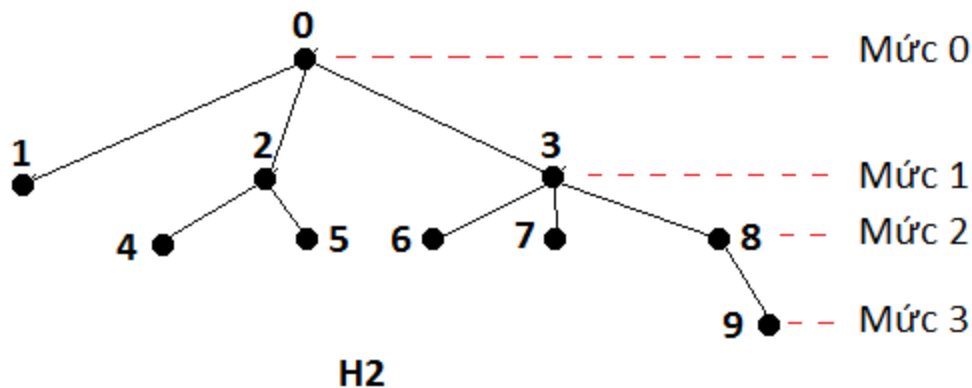


3.1.3 Định nghĩa các thuật ngữ:

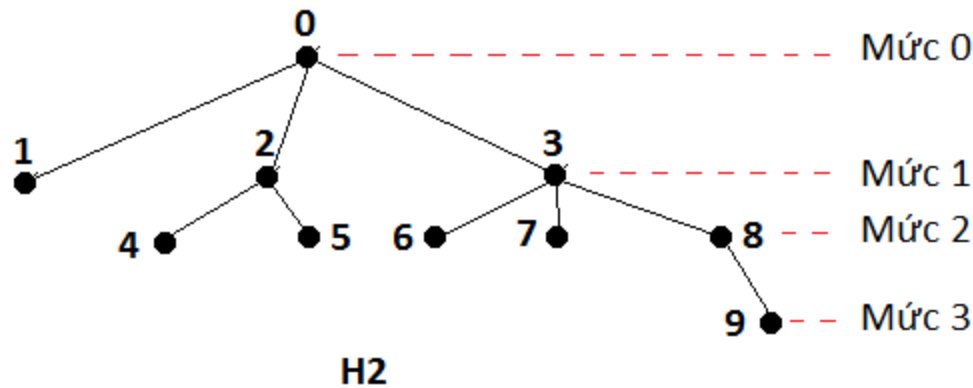
Cho T có gốc v_0 . Giả sử x, y , và z các đỉnh trong T , và $(v_0, v_1, \dots, v_{n-1}, v_n)$ đường sơ cấp trong T :

- v_{n-1} đỉnh (nút) cha của v_n .
- $v_0, v_1, \dots, v_{n-1}$ tổ tiên của v_n .
- v_n là con của v_{n-1} .
- Nếu x là tổ tiên của y , thì y là con cháu của x .
- Nếu x và y là các con của z , thì x và y anh em.
- Nếu x không có con, x là lá.
- Nếu x không là lá, x là đỉnh trong.

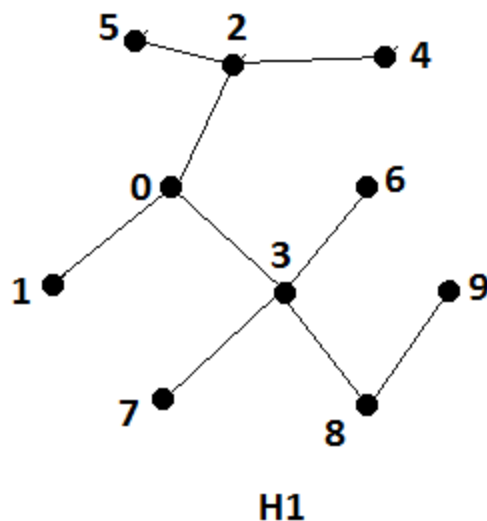
(0, 3, 8)



- **Cây con của T có gốc x** là đồ thị với tập đỉnh V và tập cạnh E , trong đó
 - + V gồm x và tất cả các con cháu của x ,
 - + $E = \{ e : e \text{ là một cạnh trên đường đi sơ cấp từ } x \text{ tới một đỉnh nào đó thuộc } V \}$.

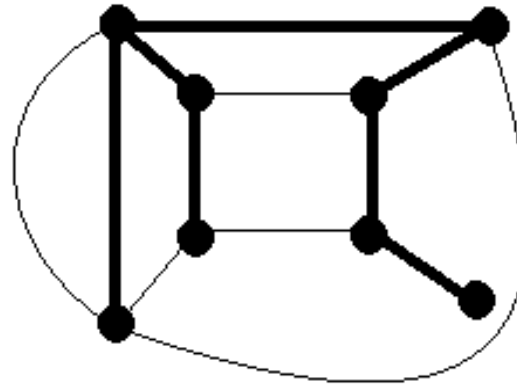
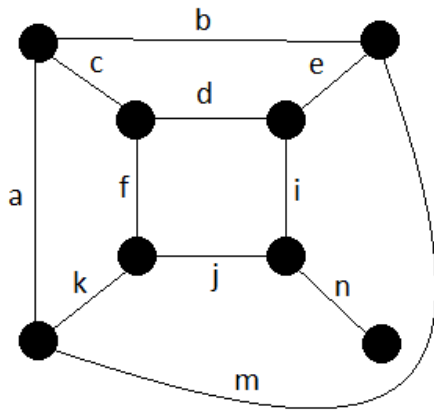


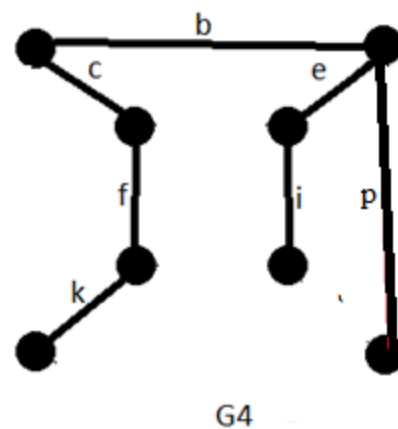
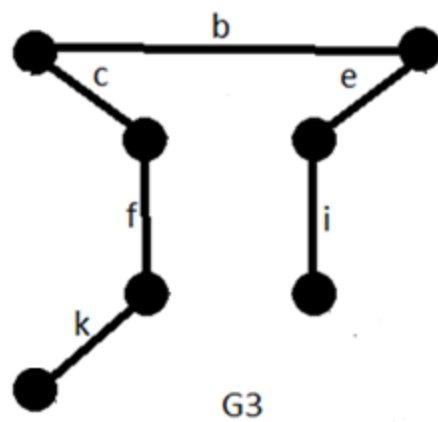
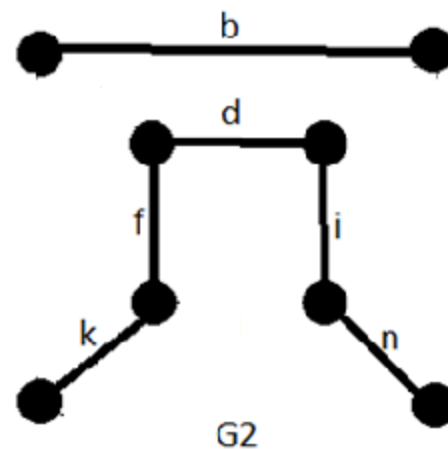
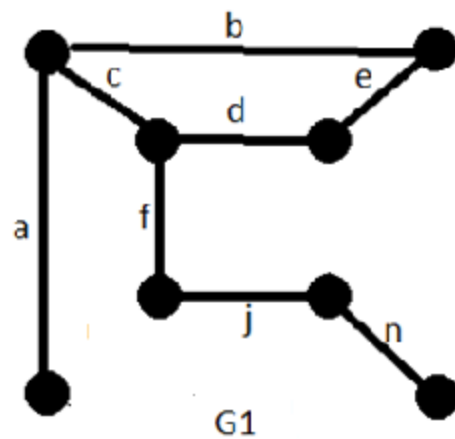
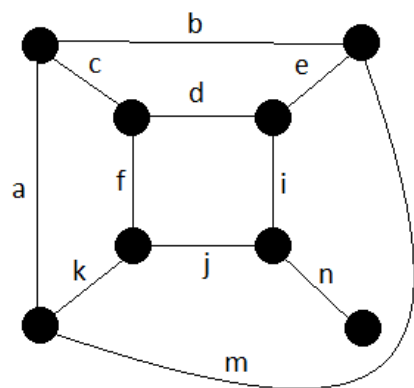
3.1.4 Định lý: Cho T là đồ thị có n đỉnh. Ta có :
 T là cây $\Leftrightarrow T$ liên thông và có $n-1$ cạnh.



3.3 Cây khung (Spanning Tree):

3.3.1 Định nghĩa : *T là cây khung của một đồ thị $G = (V, E)$ nếu T là một **cây** có tập đỉnh là tập đỉnh V và tập cạnh là tập con của E của G.*





3.3.3 Thuật toán tìm cây khung:

Thuật toán 1: **B**readth-**F**irst **S**earch tìm spanning tree

Định nghĩa hàng đợi : Hàng đợi là một tập hợp có tính chất :

- Mỗi lần cho vào hàng đợi một phần tử,
- Mỗi lần chỉ lấy một phần tử ra khỏi hàng đợi,
- Phần tử vào trước sẽ lấy ra trước,

VD : Các phần tử lần lượt đưa vào hàng đợi Q là 1, 4, 2, 3, thì thứ tự lấy ra là 1, 4, 2, 3. **đỉnh hàng đợi**.

3	2	4	1
---	---	---	---

3.3.3 Thuật toán tìm **cây khung** :

Thuật toán : **Breadth-First Search (BFS, tìm theo chiều rộng trước)**.

Gọi s là gốc. Q là hàng đợi , rỗng ban đầu.

Bước 1:

- **$i=0$**

- **Với mỗi** $u \in V$ thực hiện

$D[u] = 0$; // u chưa được duyệt

- $D[s] = 1$; // s đã được duyệt

- $p[s] = -1$;

- Thêm đỉnh s vào Q ;

Bước 2 :

i=1

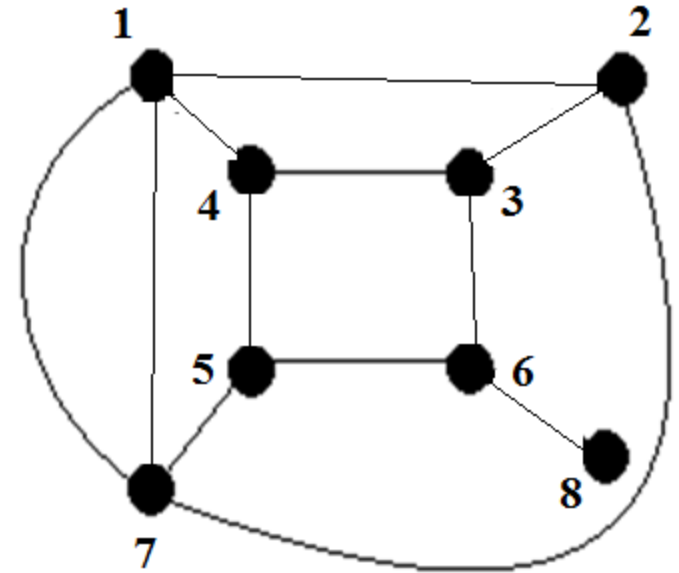
while ($Q \neq \emptyset$)

{

1. u = Phần tử đỉnh Q ; // Chỉ sao chép giá trị
 2. Với mỗi v kề u thực hiện **(Duyệt theo thứ tự)**
 3. **if** ($D[v]=0$) {
 4. $D[v]= 1$;
 5. $p[v] = u$;
 6. Thêm v vào Q ;
 - } // End If**
 7. Xóa phần tử đỉnh Q ;
 8. **i=i+1**
- } // End While**

Bước 1:

- **i=0, s = 1**
- **Với mỗi** $u \in V$ thực hiện : $D[u]= 0$;
// u chưa được duyệt
- $D[s]=1$; // s đã được duyệt
- $p[s]= -1$;
- Thêm đỉnh s vào Q;

[illegible]

Bước 2 : Duyệt theo thứ tự 1,2,3,4,5,6,7,8.

i=1;

while (Q≠∅)

{

1. u = Phần tử đỉnh Q; // Chỉ sao chép giá trị

2. Với mỗi v kề u thực hiện **3**:

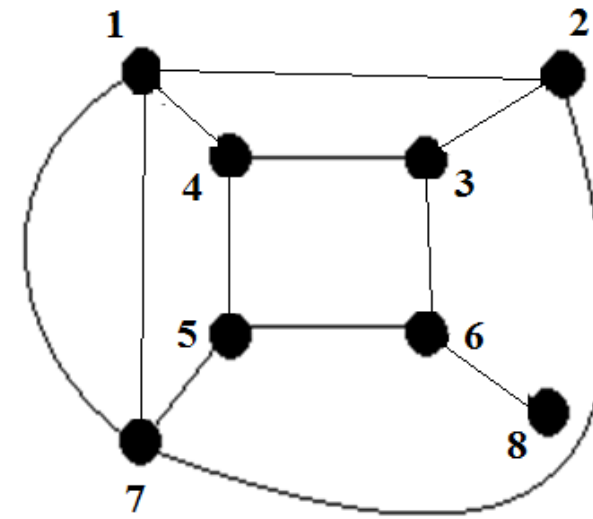
3. **if (D[v]=0) { 4. D[v]= 1; 5. p[v] = u; 6. Thêm v vào Q ; }**

7. Xóa phần tử đỉnh Q;

8. i=i+1;

} //

i	u	Q	1	2	3	4	5	6	7	8



Thuật toán : **Depth-First Search (DFS, tìm theo chiều sâu trước)**

Visit(u=1)

{

1. D[u]=1;

2. Với mỗi v kề u thực hiện :

4, 7

3. if (D[v]=0) // Chưa duyệt

{

4. p[v]=u;

5. Visit(v);

}

} // kết visit

main()

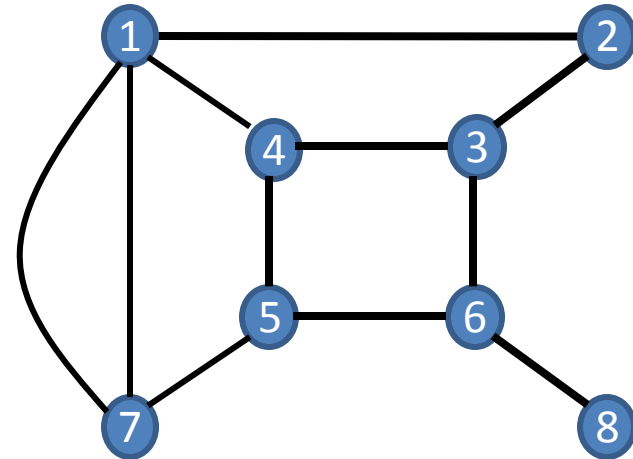
{ s=1;

Với mỗi $u \in V : D[u]=0;$

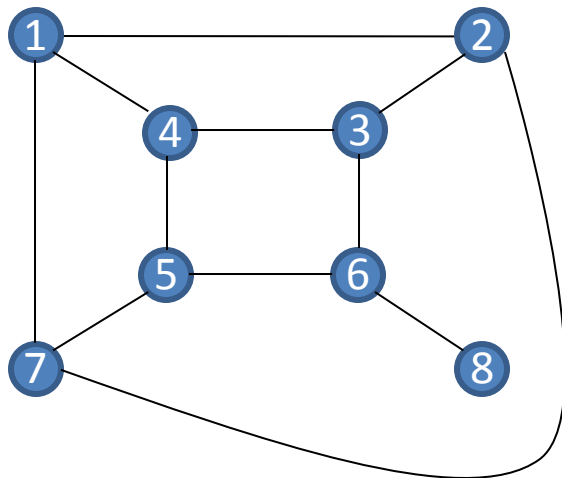
p[s]= -1;

Visit(s);

}



Bài tập : S=2. Thứ tự 12345678.

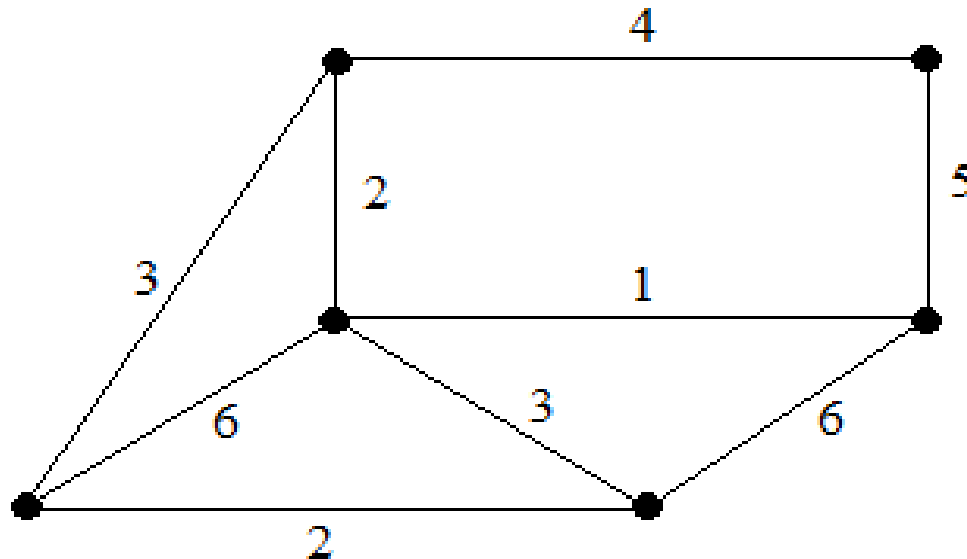


BFS

DFS

3.4 Cây khung bé nhất (Minium Spanning Tree):

3.4.1 Định nghĩa: Cho G là đồ thị có trọng số. Cây khung bé nhất là cây khung có tổng các trọng số của các cạnh là bé nhất.



3.4.2 Thuật toán: Cho G là đồ thị có trọng số, liên thông.

Thuật toán 1 : Thuật toán Kruskal .

Bước 1:

$$A = \emptyset$$

Bước 2:

Với mỗi $v \in V$

{

Tạo tập hợp chỉ có một phần tử là đỉnh v ;

}

Bước 3:

Sắp xếp các cạnh theo chiều tăng dần các trọng số.

Bước 4:

Với mỗi cạnh $e=(u,v) \in E$, theo thứ tự đã sắp, thực hiện

{

If (Tập chứa $u \neq$ Tập chứa v)

{

- $A = A \cup \{e\}$ (Thêm e vào A)

- Gộp tập chứa u và tập chứa v thành 1 tập

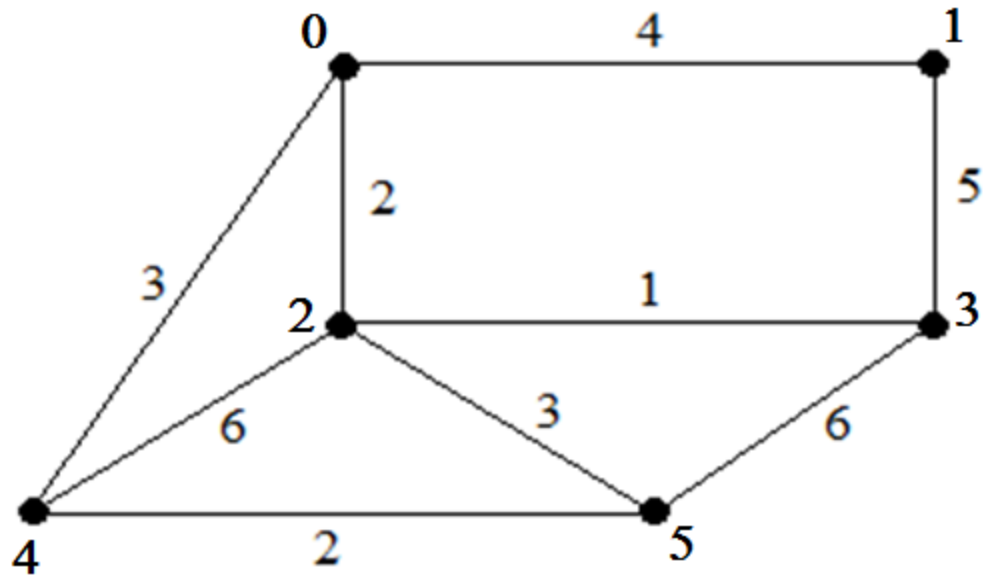
}

}

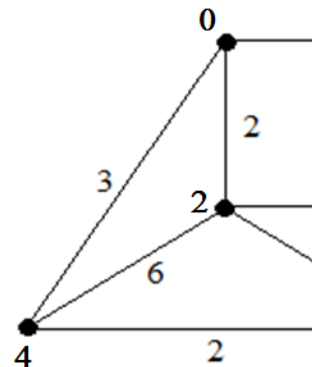
Bước 5 :

Viết A

Ví dụ: Tìm cây khung bé nhất của đồ thị



Ví dụ: Tìm cây khung bé nhất của đồ thị



Bước	Cạnh	A	Các tập
1, 2, 3	Sắp xếp ↑	$\emptyset$	$\{0\}, \{1\}, \{2\}, \{3\}, \{4\}, \{5\}$
4	(2, 3)		
	(0, 2)		
	(4, 5)		
	(0, 4)		
	(2, 5)		
	(0, 1)		
	(1, 3)		
	(2, 4)		
	(3, 6)		

Bước 4:

Với mỗi cạnh $e=(u,v) \in$
sắp, thực hiện

{

If (Tập chứa $u \neq$ Tập

{

- $A = A \cup \{e\}$ (

- Gộp tập chứa

thành 1 tập

}

}

Bước 5 :

Viết A

Thuật toán Prim :

- + Cho trước đồ thị $G=(V,E)$, V là tập các đỉnh, E là tập các cạnh
- + V_t là tập các đỉnh đã được chọn.
- + F là tập các cạnh của cây khung cực tiểu.
- + $w(u,v)$ là trọng số cạnh (u,v)

Bước 1: Khởi tạo

$$F = \{\emptyset\};$$

$$V_T = \{u\};$$

Bước 2: Xây dựng cây khung

while ($|F| < n-1$)

```
{  
    B1. Chọn  $e = \{\min w(u,v) \text{ với } (u \in V_T) \ \& \ (v \notin V_T)\}$ ;  
    B2.  $F = F \cup \{e\}$ ;  
    B3.  $V_T = V_T \cup \{v\}$ ;  
}
```

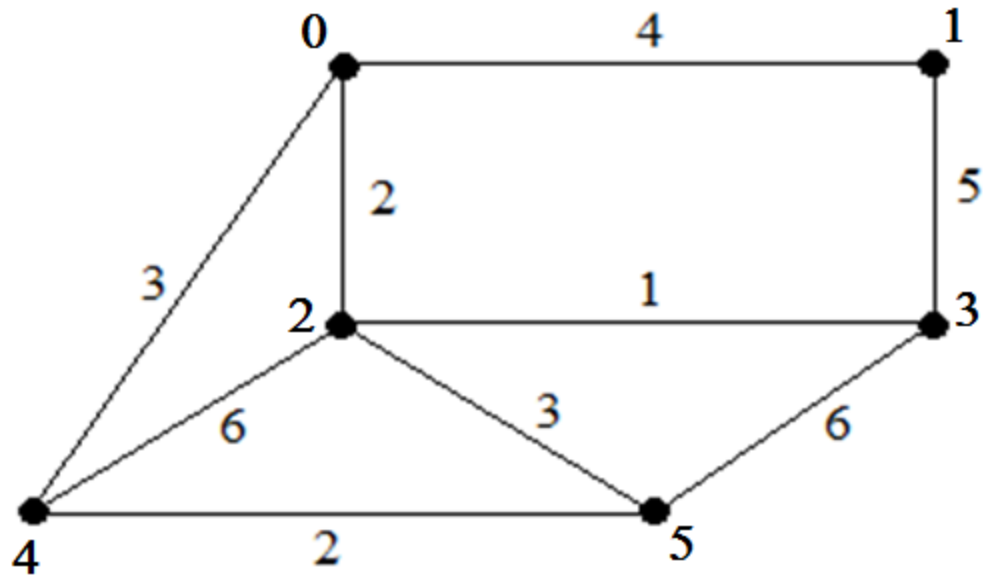
Lưu ý :

- u, v ở **B1** được chọn như sau : u là chỉ số bé nhất. Nếu có nhiều v kề u thỏa **B1** thì v bé nhất được chọn.
- Nếu các đỉnh được đánh nhãn là ký tự thì thứ tự theo thứ tự alphabet.

Thuật toán Prim .

- Đồ thị không có cạnh song song.
- r : gốc.

Ví dụ: Tìm cây khung bé nhất của đồ thị



Tài liệu tham khảo:

1. Discrete Mathematics , Richard Johnsonbaugh
2. Algorithms, Thomas h. Cormen
3. Toán Rời Rạc Nâng Cao, Trần Ngọc Danh,
ĐHQG TP HCM
4. Lý Thuyết Đồ Thị, Đặng Trường Sơn, Lê văn
Vinh, ĐHSP Kỹ Thuật TP HCM